

Graham Hutton



Programming in Haskell

Second Edition

Programming in Haskell

Second Edition

Haskell is a purely functional language that allows programmers to rapidly develop clear, concise and correct software. The language has grown in popularity in recent years, both in teaching and in industry. This book is based on the author's experience of teaching Haskell for more than 20 years. All concepts are explained from first principles and no programming experience is required, making this book accessible to a broad spectrum of readers. While Part I focuses on basic concepts, Part II introduces the reader to more advanced topics.

This new edition has been extensively updated and expanded to include recent and more advanced features of Haskell, new examples and exercises, selected solutions, and freely downloadable lecture slides and code. The presentation is clean and simple, while also being fully compliant with the latest version of the language, including recent changes concerning applicative, monadic, foldable and traversable types.

Graham Hutton is Professor of Computer Science at the University of Nottingham. He has taught Haskell to thousands of students and received numerous best lecturer awards. Hutton has served as an editor of the *Journal of Functional Programming*, chair of the Haskell Symposium and the International Conference on Functional Programming, vice-chair of the ACM Special Interest Group on Programming Languages, and he is an ACM Distinguished Scientist.

Programming in Haskell

Second Edition

GRAHAM HUTTON

University of Nottingham



CAMBRIDGE
UNIVERSITY PRESS

CAMBRIDGE

UNIVERSITY PRESS

University Printing House, Cambridge CB2 8BS, United Kingdom

One Liberty Plaza, 20th Floor, New York, NY 10006, USA
477 Williamstown Road, Port Melbourne, VIC 3207, Australia
4843/24, 2nd Floor, Ansari Road, Daryaganj, Delhi - 110002, India
79 Anson Road, #06-04/06, Singapore 079906

Cambridge University Press is part of the University of Cambridge.

It furthers the University's mission by disseminating knowledge in the pursuit of education, learning, and research at the highest international levels of excellence.

www.cambridge.org

Information on this title: www.cambridge.org/9781316626221
10.1017/9781316784099

© Graham Hutton 2007, 2016

This publication is in copyright. Subject to statutory exception and to the provisions of relevant collective licensing agreements, no reproduction of any part may take place without the written permission of Cambridge University Press.

First published 2007
Second edition 2016

Printed in the United Kingdom by Clays, St Ives plc in 2016

A catalogue record for this publication is available from the British Library

ISBN 978-1-316-62622-1 Paperback

Cambridge University Press has no responsibility for the persistence or accuracy of URLs for external or third-party Internet Web sites referred to in this publication, and does not guarantee that any content on such Web sites is, or will remain, accurate or appropriate.

For Annette, Callum and Tom

Contents

	<i>Foreword</i>	<i>page</i> xiii
	<i>Preface</i>	xv
Part I	Basic Concepts	1
1	Introduction	3
	1.1 Functions	3
	1.2 Functional programming	4
	1.3 Features of Haskell	6
	1.4 Historical background	8
	1.5 A taste of Haskell	9
	1.6 Chapter remarks	13
	1.7 Exercises	13
2	First steps	14
	2.1 Glasgow Haskell Compiler	14
	2.2 Installing and starting	14
	2.3 Standard prelude	15
	2.4 Function application	16
	2.5 Haskell scripts	17
	2.6 Chapter remarks	21
	2.7 Exercises	21
3	Types and classes	22
	3.1 Basic concepts	22
	3.2 Basic types	23
	3.3 List types	25
	3.4 Tuple types	26
	3.5 Function types	27
	3.6 Curried functions	27
	3.7 Polymorphic types	29
	3.8 Overloaded types	30
	3.9 Basic classes	31
	3.10 Chapter remarks	36

3.11	Exercises	36
4	Defining functions	38
4.1	New from old	38
4.2	Conditional expressions	38
4.3	Guarded equations	39
4.4	Pattern matching	40
4.5	Lambda expressions	42
4.6	Operator sections	44
4.7	Chapter remarks	45
4.8	Exercises	45
5	List comprehensions	47
5.1	Basic concepts	47
5.2	Guards	48
5.3	The <code>zip</code> function	50
5.4	String comprehensions	51
5.5	The Caesar cipher	52
5.6	Chapter remarks	56
5.7	Exercises	57
6	Recursive functions	59
6.1	Basic concepts	59
6.2	Recursion on lists	61
6.3	Multiple arguments	63
6.4	Multiple recursion	64
6.5	Mutual recursion	65
6.6	Advice on recursion	66
6.7	Chapter remarks	71
6.8	Exercises	71
7	Higher-order functions	73
7.1	Basic concepts	73
7.2	Processing lists	74
7.3	The <code>foldr</code> function	76
7.4	The <code>foldl</code> function	79
7.5	The composition operator	81
7.6	Binary string transmitter	82
7.7	Voting algorithms	86
7.8	Chapter remarks	89
7.9	Exercises	89
8	Declaring types and classes	92
8.1	Type declarations	92

8.2	Data declarations	93
8.3	Newtype declarations	95
8.4	Recursive types	96
8.5	Class and instance declarations	99
8.6	Tautology checker	101
8.7	Abstract machine	106
8.8	Chapter remarks	108
8.9	Exercises	109
9	The countdown problem	111
9.1	Introduction	111
9.2	Arithmetic operators	112
9.3	Numeric expressions	113
9.4	Combinatorial functions	114
9.5	Formalising the problem	115
9.6	Brute force solution	115
9.7	Performance testing	116
9.8	Combining generation and evaluation	117
9.9	Exploiting algebraic properties	118
9.10	Chapter remarks	119
9.11	Exercises	120
Part II	Going Further	121
10	Interactive programming	123
10.1	The problem	123
10.2	The solution	124
10.3	Basic actions	125
10.4	Sequencing	126
10.5	Derived primitives	127
10.6	Hangman	128
10.7	Nim	129
10.8	Life	133
10.9	Chapter remarks	137
10.10	Exercises	137
11	Unbeatable tic-tac-toe	139
11.1	Introduction	139
11.2	Basic declarations	140
11.3	Grid utilities	141
11.4	Displaying a grid	142
11.5	Making a move	143
11.6	Reading a number	144
11.7	Human vs human	144

11.8	Game trees	145
11.9	Pruning the tree	147
11.10	Minimax algorithm	148
11.11	Human vs computer	150
11.12	Chapter remarks	151
11.13	Exercises	151
12	Monads and more	153
12.1	Functors	153
12.2	Applicatives	157
12.3	Monads	164
12.4	Chapter remarks	174
12.5	Exercises	175
13	Monadic parsing	177
13.1	What is a parser?	177
13.2	Parsers as functions	177
13.3	Basic definitions	179
13.4	Sequencing parsers	179
13.5	Making choices	181
13.6	Derived primitives	183
13.7	Handling spacing	186
13.8	Arithmetic expressions	187
13.9	Calculator	191
13.10	Chapter remarks	194
13.11	Exercises	194
14	Foldables and friends	196
14.1	Monoids	196
14.2	Foldables	200
14.3	Traversables	206
14.4	Chapter remarks	210
14.5	Exercises	210
15	Lazy evaluation	212
15.1	Introduction	212
15.2	Evaluation strategies	213
15.3	Termination	216
15.4	Number of reductions	217
15.5	Infinite structures	219
15.6	Modular programming	220
15.7	Strict application	223
15.8	Chapter remarks	226
15.9	Exercises	226

16	Reasoning about programs	228
16.1	Equational reasoning	228
16.2	Reasoning about Haskell	229
16.3	Simple examples	230
16.4	Induction on numbers	231
16.5	Induction on lists	234
16.6	Making append vanish	238
16.7	Compiler correctness	241
16.8	Chapter remarks	246
16.9	Exercises	246
17	Calculating compilers	249
17.1	Introduction	249
17.2	Syntax and semantics	249
17.3	Adding a stack	250
17.4	Adding a continuation	252
17.5	Defunctionalising	254
17.6	Combining the steps	257
17.7	Chapter remarks	261
17.8	Exercises	261
Appendix A	Selected solutions	263
A.1	Introduction	263
A.2	First steps	264
A.3	Types and classes	265
A.4	Defining functions	266
A.5	List comprehensions	267
A.6	Recursive functions	267
A.7	Higher-order functions	268
A.8	Declaring types and classes	269
A.9	The countdown problem	270
A.10	Interactive programming	270
A.11	Unbeatable tic-tac-toe	271
A.12	Monads and more	272
A.13	Monadic parsing	273
A.14	Foldables and friends	274
A.15	Lazy evaluation	275
A.16	Reasoning about programs	276
A.17	Calculating compilers	279
Appendix B	Standard prelude	280
B.1	Basic classes	280
B.2	Booleans	281
B.3	Characters	282

B.4	Strings	283
B.5	Numbers	283
B.6	Tuples	284
B.7	Maybe	284
B.8	Lists	285
B.9	Functions	287
B.10	Input/output	287
B.11	Functors	288
B.12	Applicatives	289
B.13	Monads	290
B.14	Alternatives	290
B.15	MonadPlus	291
B.16	Monoids	292
B.17	Foldables	294
B.18	Traversables	297
	<i>Bibliography</i>	298
	<i>Index</i>	300

Foreword

It is nearly a century ago that Alonzo Church introduced the lambda calculus, and over half a century ago that John McCarthy introduced Lisp, the world's second oldest programming language and the first functional language based on the lambda calculus. By now, every major programming language including JavaScript, C++, Swift, Python, PHP, Visual Basic, Java, . . . has support for lambda expressions or anonymous higher-order functions.

As with any idea that becomes mainstream, inevitably the underlying foundations and principles get watered down or forgotten. Lisp allowed mutation, yet today many confuse functions as first-class citizens with immutability. At the same time, other effects such as exceptions, reflection, communication with the outside world, and concurrency go unmentioned. Adding recursion in the form of feedback-loops to pure combinational circuits lets us implement mutable state via flip-flops. Similarly, using one effect such as concurrency or input/output we can simulate other effects such as mutability. John Hughes famously stated in his classic paper *Why Functional Programming Matters* that we cannot make a language more powerful by eliminating features. To that, we add that often we cannot even make a language less powerful by removing features. In this book, Graham demonstrates convincingly that the true value of functional programming lies in leveraging first-class functions to achieve compositionality and equational reasoning. Or in Graham's own words, "functional programming can be viewed as a style of programming in which the basic method of computation is the application of functions to arguments". These functions do not necessarily have to be pure or statically typed in order to realise the simplicity, elegance, and conciseness of expression that we get from the functional style.

While you can code like a functional hacker in a plethora of languages, a semantically pure and lazy, and syntactically lean and terse language such as Haskell is still the best way to learn how to think like a fundamentalist. Based upon decades of teaching experience, and backed by an impressive stream of research papers, in this book Graham gently guides us through the whole gambit of key functional programming concepts such as higher-order functions, recursion, list comprehensions, algebraic datatypes and pattern matching. The book does not shy away from more advanced concepts. If you are still confused by the n-th blog post that attempts to explain monads, you are in the right place. Gently starting with the IO monad, Graham progresses from functors to applicatives using many concrete examples. By the time he arrives at monads, every reader will feel that they themselves could have come up with the concept of a monad as a generic pattern for composing functions with effects. The chapter on monadic

parsers brings everything together in a compelling use-case of parsing arithmetic expressions in the implementation of a simple calculator.

This new edition not only adds many more concrete examples of concepts introduced throughout the book, it also introduces the novel Haskell concepts of foldable and traversable types. Readers familiar with object-oriented languages routinely use iterables and visitors to enumerate over all values in a container, or respectively to traverse complex data structures. Haskell's higher-kinded type classes allow for a very concise and abstract treatment of these concepts by means of the Foldable and Traversable classes. Last but not least, the final chapters of the book give an in-depth overview of lazy evaluation and equational reasoning to prove and derive programs. The capstone chapter on calculating compilers especially appeals to me because it touches a topic that has had my keen interest for many decades, ever since my own PhD thesis on the same topic.

While there are plenty of alternative textbooks on Haskell in particular and functional programming in general, Graham's book is unique amongst all of these in that it uses Haskell simply as a tool for thought, and never attempts to sell Haskell or functional programming as a silver bullet that magically solves all programming problems. It focuses on elegant and concise expression of intent and thus makes a strong case of how pure and lazy functional programming is an intelligible medium for efficiently reasoning about algorithms at a high level of abstraction. The skills you acquire by studying this book will make you a much better programmer no matter what language you use to actually program in. In the past decade, using the first edition of this book I have taught many tens of thousands of students how to juggle with code. With this new edition, I am looking forward to extending this streak for at least another 10 years.

Erik Meijer

Preface

What is this book?

Haskell is a purely functional language that allows programmers to rapidly develop software that is clear, concise and correct. The book is aimed at a broad spectrum of readers who are interested in learning the language, including professional programmers, university students and high-school students. However, no programming experience is required or assumed, and all concepts are explained from first principles with the aid of carefully chosen examples and exercises. Most of the material in the book should be accessible to anyone over the age of around sixteen with a reasonable aptitude for scientific ideas.

How is it structured?

The book is divided into two parts. Part I introduces the basic concepts of pure programming in Haskell and is structured around the core features of the language, such as types, functions, list comprehensions, recursion and higher-order functions. Part II covers impure programming and a range of more advanced topics, such as monads, parsing, foldable types, lazy evaluation and reasoning about programs. The book contains many extended programming examples, and each chapter includes suggestions for further reading and a series of exercises. The appendices provide solutions to selected exercises, and a summary of some of the most commonly used definitions from the Haskell standard prelude.

What is its approach?

The book aims to teach the key concepts of Haskell in a clean and simple manner. As this is a textbook rather than a reference manual we do not attempt to cover all aspects of the language and its libraries, and we sometimes choose to define functions from first principles rather than using library functions. As the book progresses the level of generality that is used is gradually increased. For example, in the beginning most of the functions that are used are specialised to simple types, and later on we see how many functions can be generalised to larger classes of types by exploiting particular features of Haskell.

How should it be read?

The basic material in part I can potentially be worked through fairly quickly, particularly for those with some prior programming experience, but additional time and effort may be required to absorb some of material in part II. Readers are recommended to work through all the material in part I, and then select

appropriate material from part II depending on their own interests. It is vital to write Haskell code for yourself as you go along, as you can't learn to program just by reading. Try out the examples from each chapter as you proceed, and solve the exercises for each chapter before checking the solutions.

What's new in this edition?

The book is an extensively revised and expanded version of the first edition. It has been extended with new chapters that cover more advanced aspects of Haskell, new examples and exercises to further reinforce the concepts being introduced, and solutions to selected exercises. The remaining material has been completely reworked in response to changes in the language and feedback from readers. The new edition uses the Glasgow Haskell Compiler (GHC), and is fully compatible with the latest version of the language, including recent changes concerning applicative, monadic, foldable and traversable types.

How can it be used for teaching?

An introductory course might cover all of part I and a few selected topics from part II; my first-year course covers chapters 1–9, 10 and 15. An advanced course might start with a refresher of part I, and cover a selection of more advanced topics from part II; my second-year course focuses on chapters 12 and 16, and is taught interactively on the board. The website for the book provides a range of supporting materials, including PowerPoint slides and Haskell code for the extended examples. Instructors can obtain a large collection of exams and solutions based on material in the book from solutions@cambridge.org.

Acknowledgements

I am grateful to the University of Nottingham for providing a sabbatical to produce this new edition; Thorsten Altenkirch, Venanzio Capretta, Henrik Nilsson and other members of the FP lab for our many enjoyable discussions; Iván Pérez Domínguez for useful comments on a number of chapters; the students and tutors on all of my Haskell courses for their feedback; Clare Dennison, David Tranah and Abigail Walkington at CUP for their editorial work; the GHC team for producing such a great compiler; and finally, Catherine and Ian Hutton for getting me started in computing all those years ago.

Many thanks also to Ki Yung Ahn, Bob Davison, Philip Hölzenspies and Neil Mitchell for providing detailed comments on the first edition, and to the following for pointing out errors and typos: Paul Brown, Sergio Queiroz de Medeiros, David Duke, Robert Fabian, Ben Fleis, Robert Furber, Andrew Kish, Tomoyas Kobayashi, Florian Larysch, Carlos Oroz, Douglas Philips, Bruce Turner, Gregor Ulm, Marco Valtorta and Kazu Yamamoto. All of these comments have been taken into account when preparing the new edition.

Graham Hutton

Part I

Basic Concepts

1 Introduction

In this chapter we set the stage for the rest of the book. We start by reviewing the notion of a function, then introduce the concept of functional programming, summarise the main features of Haskell and its historical background, and conclude with three small examples that give a taste of Haskell.

1.1 Functions

In Haskell, a *function* is a mapping that takes one or more arguments and produces a single result, and is defined using an equation that gives a name for the function, a name for each of its arguments, and a body that specifies how the result can be calculated in terms of the arguments.

For example, a function `double` that takes a number `x` as its argument, and produces the result `x + x`, can be defined by the following equation:

```
double x = x + x
```

When a function is applied to actual arguments, the result is obtained by substituting these arguments into the body of the function in place of the argument names. This process may immediately produce a result that cannot be further simplified, such as a number. More commonly, however, the result will be an expression containing other function applications, which must then be processed in the same way to produce the final result.

For example, the result of the application `double 3` of the function `double` to the number `3` can be determined by the following calculation, in which each step is explained by a short comment in curly parentheses:

```
double 3
=      { applying double }
  3 + 3
=      { applying + }
  6
```

Similarly, the result of the nested application `double (double 2)` in which the function `double` is applied twice can be calculated as follows:

```
double (double 2)
```

```

=      { applying the inner double }
      double (2 + 2)
=      { applying + }
      double 4
=      { applying double }
      4 + 4
=      { applying + }
      8

```

Alternatively, the same result can also be calculated by starting with the outer application of the function `double` rather than the inner:

```

      double (double 2)
=      { applying the outer double }
      double 2 + double 2
=      { applying the first double }
      (2 + 2) + double 2
=      { applying the first + }
      4 + double 2
=      { applying double }
      4 + (2 + 2)
=      { applying the second + }
      4 + 4
=      { applying + }
      8

```

However, this approach requires two more steps than our original version, because the expression `double 2` is duplicated in the first step and hence simplified twice. In general, the order in which functions are applied in a calculation does not affect the value of the final result, but it may affect the number of steps required, and whether the calculation process terminates. These issues are explored in more detail when we consider how expressions are evaluated in chapter 15.

1.2 Functional programming

What is functional programming? Opinions differ, and it is difficult to give a precise definition. Generally speaking, however, functional programming can be viewed as a *style* of programming in which the basic method of computation is the application of functions to arguments. In turn, a functional programming language is one that *supports* and *encourages* the functional style.

To illustrate these ideas, let us consider the task of computing the sum of the integers (whole numbers) between one and some larger number `n`. In many current programming languages, this would normally be achieved using two integer variables whose values can be changed over time by means of the assignment

operator `=`, with one such variable used to accumulate the total, and the other used to count from 1 to `n`. For example, in Java the following program computes the required sum using this approach:

```
int total = 0;
for (int count = 1; count <= n; count++)
    total = total + count;
```

That is, we first initialise an integer variable `total` to zero, and then enter a loop that ranges an integer variable `count` from 1 to `n`, adding the current value of the counter to the total each time round the loop.

In the above program, the basic method of computation is *changing stored values*, in the sense that executing the program results in a sequence of assignments. For example, the case of `n = 5` gives the following sequence, in which the final value assigned to the variable `total` is the required sum:

```
total = 0;
count = 1;
total = 1;
count = 2;
total = 3;
count = 3;
total = 6;
count = 4;
total = 10;
count = 5;
total = 15;
```

In general, programming languages such as Java in which the basic method of computation is changing stored values are called *imperative* languages, because programs in such languages are constructed from imperative instructions that specify precisely how the computation should proceed.

Now let us consider computing the sum of the numbers between one and `n` using Haskell. This would normally be achieved using two library functions, one called `[..]` that is used to produce the list of numbers between 1 and `n`, and the other called `sum` that is used to produce the sum of this list:

```
sum [1..n]
```

In this program, the basic method of computation is *applying functions to arguments*, in the sense that executing the program results in a sequence of applications. For example, the case of `n = 5` gives the following sequence, in which the final value in the sequence is the required sum:

```
sum [1..5]
=      { applying [..] }
sum [1,2,3,4,5]
```

$$\begin{aligned} &= \quad \{ \text{applying } \texttt{sum} \} \\ &\quad 1 + 2 + 3 + 4 + 5 \\ &= \quad \{ \text{applying } + \} \\ &\quad 15 \end{aligned}$$

Most imperative languages provide some form of support for programming with functions, so the Haskell program `sum [1..n]` could be translated into such languages. However, many imperative languages do not *encourage* programming in the functional style. For example, many such languages discourage or prohibit functions from being stored in data structures such as lists, from constructing intermediate structures such as the list of numbers in the above example, from taking functions as arguments or producing functions as results, or from being defined in terms of themselves. In contrast, Haskell imposes no such restrictions on how functions can be used, and provides a range of features to make programming with functions both simple and powerful.

1.3 Features of Haskell

For reference, the main features of Haskell are listed below, along with particular chapters of this book that give further details.

- **Concise programs** (chapters 2 and 4)

Due to the high-level nature of the functional style, programs written in Haskell are often much more *concise* than programs written in other languages, as illustrated by the example in the previous section. Moreover, the syntax of Haskell has been designed with concise programs in mind, in particular by having few keywords, and by allowing indentation to be used to indicate the structure of programs. Although it is difficult to make an objective comparison, Haskell programs are often between two and ten times shorter than programs written in other languages.

- **Powerful type system** (chapters 3 and 8)

Most modern programming languages include some form of *type system* to detect incompatibility errors, such as erroneously attempting to add a number and a character. Haskell has a type system that usually requires little type information from the programmer, but allows a large class of incompatibility errors in programs to be automatically detected prior to their execution, using a sophisticated process called type inference. The Haskell type system is also more powerful than most languages, supporting very general forms of *polymorphism* and *overloading*, and providing a wide range of special purpose features concerning types.

- **List comprehensions** (chapter 5)

One of the most common ways to structure and manipulate data in computing is using lists of values. To this end, Haskell provides lists as a basic

concept in the language, together with a simple but powerful *comprehension* notation that constructs new lists by selecting and filtering elements from one or more existing lists. Using the comprehension notation allows many common functions on lists to be defined in a clear and concise manner, without the need for explicit recursion.

- **Recursive functions** (chapter 6)

Most programs involve some form of looping. In Haskell, the basic mechanism by which looping is achieved is through *recursive* functions that are defined in terms of themselves. It can take some time to get used to recursion, particularly for those with experience of programming in other styles. But as we shall see, many computations have a simple and natural definition in terms of recursive functions, especially when *pattern matching* and *guards* are used to separate different cases into different equations.

- **Higher-order functions** (chapter 7)

Haskell is a *higher-order* functional language, which means that functions can freely take functions as arguments and produce functions as results. Using higher-order functions allows common programming patterns, such as composing two functions, to be defined as functions within the language itself. More generally, higher-order functions can be used to define *domain-specific languages* within Haskell itself, such as for list processing, interactive programming, and parsing.

- **Effectful functions** (chapters 10 and 12)

Functions in Haskell are pure functions that take all their inputs as arguments and produce all their outputs as results. However, many programs require some form of *side effect* that would appear to be at odds with purity, such as reading input from the keyboard, or writing output to the screen, while the program is running. Haskell provides a uniform framework for programming with effects, without compromising the purity of functions, based upon the use of *monads* and *applicatives*.

- **Generic functions** (chapters 12 and 14)

Most languages allow functions to be defined that are *generic* over a range of simple types, such as different forms of numbers. However, the Haskell type system also supports functions that are generic over much richer kinds of structures. For example, the language provides a range of library functions that can be used with any type that is *functorial*, *applicative*, *monadic*, *foldable*, or *traversable*, and moreover, allows new structures and generic functions over them to be defined.

- **Lazy evaluation** (chapter 15)

Haskell programs are executed using a technique called *lazy evaluation*, which is based upon the idea that no computation should be performed

until its result is actually required. As well as avoiding unnecessary computation, lazy evaluation ensures that programs terminate whenever possible, encourages programming in a modular style using intermediate data structures, and even allows programming with infinite structures.

- **Equational reasoning** (chapters 16 and 17)

Because programs in Haskell are pure functions, simple *equational reasoning* techniques can be used to execute programs, to transform programs, to prove properties of programs, and even to calculate programs directly from specifications of their intended behaviour. Equational reasoning is particularly powerful when combined with the use of *induction* to reason about functions that are defined using recursion.

1.4 Historical background

Many of the features of Haskell are not new, but were first introduced by other languages. To help place Haskell in context, some of the key historical developments related to the language are briefly summarised below:

- In the 1930s, Alonzo Church developed the lambda calculus, a simple but powerful mathematical theory of functions.
- In the 1950s, John McCarthy developed Lisp (“LISt Processor”), generally regarded as being the first functional programming language. Lisp had some influences from the lambda calculus, but still retained the concept of variable assignment as a central feature of the language.
- In the 1960s, Peter Landin developed ISWIM (“If you See What I Mean”), the first pure functional programming language, based strongly on the lambda calculus and having no variable assignments.
- In the 1970s, John Backus developed FP (“Functional Programming”), a functional programming language that particularly emphasised the idea of higher-order functions and reasoning about programs.
- Also in the 1970s, Robin Milner and others developed ML (“Meta-Language”), the first of the modern functional programming languages, which introduced the idea of polymorphic types and type inference.
- In the 1970s and 1980s, David Turner developed a number of lazy functional programming languages, culminating in the commercially produced language Miranda (meaning “admirable”).
- In 1987, an international committee of programming language researchers initiated the development of Haskell (named after the logician Haskell Curry), a standard lazy functional programming language.

- In the 1990s, Philip Wadler and others developed the concept of type classes to support overloading, and the use of monads to handle effects, two of the main innovative features of Haskell.
- In 2003, the Haskell committee published the Haskell Report, which defined a long-awaited stable version of the language.
- In 2010, a revised and updated of the Haskell Report was published. Since then the language has continued to evolve, in response to both new foundational developments and new practical experience.

It is worthy of note that three of the above individuals — McCarthy, Backus, and Milner — have each received the ACM Turing Award, which is generally regarded as being the computing equivalent of a Nobel prize.

1.5 A taste of Haskell

We conclude this chapter with three small examples that give a taste of programming in Haskell. The examples involve processing lists of values of different types, and illustrate different features of the language.

Summing numbers

Recall the function `sum` used earlier in this chapter, which produces the sum of a list of numbers. In Haskell, `sum` can be defined using two equations:

```
sum []      = 0
sum (n:ns) = n + sum ns
```

The first equation states that the sum of the empty list is zero, while the second states that the sum of any non-empty list comprising a first number `n` and a remaining list of numbers `ns` is given by adding `n` and the sum of `ns`. For example, the result of `sum [1,2,3]` can be calculated as follows:

```
sum [1,2,3]
=      { applying sum }
  1 + sum [2,3]
=      { applying sum }
  1 + (2 + sum [3])
=      { applying sum }
  1 + (2 + (3 + sum []))
=      { applying sum }
  1 + (2 + (3 + 0))
=      { applying + }
  6
```

Note that even though the function `sum` is defined in terms of itself and is hence *recursive*, it does not loop forever. In particular, each application of `sum` reduces the length of the argument list by one, until the list eventually becomes empty, at which point the recursion stops and the additions are performed. Returning zero as the sum of the empty list is appropriate because zero is the *identity* for addition. That is, $0 + x = x$ and $x + 0 = x$ for any number x .

In Haskell, every function has a *type* that specifies the nature of its arguments and results, which is automatically inferred from the definition of the function. For example, the function `sum` defined above has the following type:

```
Num a => [a] -> a
```

This type states that for any type `a` of numbers, `sum` is a function that maps a list of such numbers to a single such number. Haskell supports many different types of numbers, including integers such as `123`, and floating-point numbers such as `3.14159`. Hence, for example, `sum` could be applied to a list of integers, as in the calculation above, or to a list of floating-point numbers.

Types provide useful information about the nature of functions, but, more importantly, their use allows many errors in programs to be automatically detected prior to executing the programs themselves. In particular, for every occurrence of function application in a program, a check is made that the type of the actual arguments is compatible with the type of the function itself. For example, attempting to apply the function `sum` to a list of characters would be reported as an error, because characters are not a type of numbers.

Sorting values

Now let us consider a more sophisticated function concerning lists, which illustrates a number of other aspects of Haskell. Suppose that we define a function called `qsort` by the following two equations:

```
qsort []      = []
qsort (x:xs) = qsort smaller ++ [x] ++ qsort larger
               where
                 smaller = [a | a <- xs, a <= x]
                 larger  = [b | b <- xs, b > x]
```

In this definition, `++` is an operator that appends two lists together; for example, `[1,2,3] ++ [4,5] = [1,2,3,4,5]`. In turn, `where` is a keyword that introduces local definitions, in this case a list `smaller` comprising all elements `a` from the list `xs` that are less than or equal to `x`, together with a list `larger` comprising all elements `b` from `xs` that are greater than `x`. For example, if `x = 3` and `xs = [5,1,4,2]`, then `smaller = [1,2]` and `larger = [5,4]`.

What does `qsort` actually do? First of all, we note that it has no effect on lists with a single element, in the sense that `qsort [x] = [x]` for any `x`. It is easy to verify this property using a simple calculation:

```

    qsort [x]
=      { applying qsort }
    qsort [] ++ [x] ++ qsort []
=      { applying qsort }
    [] ++ [x] ++ []
=      { applying ++ }
    [x]

```

In turn, we now work through the application of `qsort` to an example list, using the above property to simplify the calculation:

```

    qsort [3,5,1,4,2]
=      { applying qsort }
    qsort [1,2] ++ [3] ++ qsort [5,4]
=      { applying qsort }
    (qsort [] ++ [1] ++ qsort [2]) ++ [3]
      ++ (qsort [4] ++ [5] ++ qsort [])
=      { applying qsort, above property }
    ([1] ++ [2]) ++ [3] ++ ([4] ++ [5] ++ [])
=      { applying ++ }
    [1,2] ++ [3] ++ [4,5]
=      { applying ++ }
    [1,2,3,4,5]

```

In summary, `qsort` has sorted the example list into numerical order. More generally, this function produces a sorted version of any list of numbers. The first equation for `qsort` states that the empty list is already sorted, while the second states that any non-empty list can be sorted by inserting the first number between the two lists that result from sorting the remaining numbers that are **smaller** and **larger** than this number. This method of sorting is called *quicksort*, and is one of the best such methods known.

The above implementation of quicksort is an excellent example of the power of Haskell, being both clear and concise. Moreover, the function `qsort` is also more general than might be expected, being applicable not just with numbers, but with any type of ordered values. More precisely, the type

```
qsort :: Ord a => [a] -> [a]
```

states that, for any type `a` of ordered values, `qsort` is a function that maps between lists of such values. Haskell supports many different types of ordered values, including numbers, single characters such as `'a'`, and strings of characters such as `"abcde"`. Hence, for example, the function `qsort` could also be used to sort a list of characters, or a list of strings.

Sequencing actions

Our third and final example further emphasises the level of precision and generality that can be achieved in Haskell. Consider a function called `seqn` that takes a list of input/output actions, such as reading or writing a single character, performs each of these actions in sequence, and returns a list of resulting values. In Haskell, this function can be defined as follows:

```
seqn []          = return []
seqn (act:acts) = do x <- act
                  xs <- seqn acts
                  return (x:xs)
```

These two equations state that if the list of actions is empty we return the empty list of results, otherwise we perform the first action in the list, then perform the remaining actions, and finally return the list of results that were produced. For example, the expression `seqn [getChar,getChar,getChar]` reads three characters from the keyboard using the action `getChar` that reads a single character, and returns a list containing the three characters.

The interesting aspect of the function `seqn` is its type. One possible type that can be inferred from the above definition is the following:

```
seqn :: [IO a] -> IO [a]
```

This type states that `seqn` maps a list of `IO` (input/output) actions that produce results of some type `a` to a single `IO` action that produces a list of such results, which captures the high-level behaviour of `seqn` in a clear and concise manner. More importantly, however, the type also makes explicit that the function `seqn` involves the *side effect* of performing input/output actions. Using types in this manner to keep a clear distinction between functions that are pure and those that involve side effects is a central aspect of Haskell, and brings important benefits in terms of both programming and reasoning.

In fact, the function `seqn` is more general than it may initially appear. In particular, the manner in which the function is defined is not specific to the case of input/output actions, but is equally valid for other forms of effects too. For example, it can also be used to sequence actions that may change stored values, fail to succeed, write to a log file, and so on. This flexibility is captured in Haskell by means of the following more general type:

```
seqn :: Monad m => [m a] -> m [a]
```

That is, for any *monadic* type `m`, of which `IO` is just one example, `seqn` maps a list of actions of type `m a` into a single action that returns a list of values of type `a`. Being able to define generic functions such as `seqn` that can be used with different kinds of effects is a key feature of Haskell.

1.6 Chapter remarks

The Haskell Report is freely available from <http://www.haskell.org>. More detailed historical accounts of the development of functional languages in general, and Haskell in particular, are given in [1] and [2].

1.7 Exercises

1. Give another possible calculation for the result of `double (double 2)`.
2. Show that `sum [x] = x` for any number `x`.
3. Define a function `product` that produces the product of a list of numbers, and show using your definition that `product [2,3,4] = 24`.
4. How should the definition of the function `qsort` be modified so that it produces a *reverse* sorted version of a list?
5. What would be the effect of replacing `<=` by `<` in the original definition of `qsort`? Hint: consider the example `qsort [2,2,3,1,1]`.

Solutions to exercises 1–3 are given in appendix A.

2 First steps

In this chapter we take our first proper steps with Haskell. We start by introducing the GHC system and the standard prelude, then explain the notation for function application, develop our first Haskell script, and conclude by discussing a number of syntactic conventions concerning scripts.

2.1 Glasgow Haskell Compiler

As we saw in the previous chapter, small Haskell programs can be executed by hand. In practice, however, we usually require a system that can execute programs automatically. In this book we use the *Glasgow Haskell Compiler*, a state-of-the-art, open source implementation of Haskell.

The system has two main components: a batch compiler called GHC, and an interactive interpreter called GHCi. We will primarily use the interpreter in this book, as its interactive nature makes it well suited for teaching and prototyping purposes, and its performance is sufficient for most of our applications. However, if greater performance or a stand-alone executable version of a Haskell program is required, the compiler itself can be used. For example, we will use the compiler in extended programming examples in chapters 9 and 11.

2.2 Installing and starting

The Glasgow Haskell Compiler is freely available for a range of operating systems from the Haskell home page, <http://www.haskell.org>. For first time users we recommend downloading the *Haskell Platform*, which provides a convenient means to install the system and a collection of commonly used libraries. More advanced users may prefer to install the system and libraries manually.

Once installed, the interactive GHCi system can be started from the terminal command prompt, such as \$, by simply typing `ghci`:

```
$ ghci
```

All being well, a welcome message will then be displayed:

```
GHCi, version A.B.C: http://www.haskell.org/ghc/ :? for help
Prelude>
```

The GHCi prompt `>` indicates that the system is now waiting for the user to enter an expression to be evaluated. For example, it can be used as a calculator to evaluate simple numeric expressions:

```
> 2+3*4
14

> (2+3)*4
20

> sqrt (3^2 + 4^2)
5.0
```

Following normal mathematical convention, in Haskell exponentiation is assumed to have higher priority than multiplication and division, which in turn have higher priority than addition and subtraction. For example, $2*3^4$ means $2*(3^4)$, while $2+3*4$ means $2+(3*4)$. Moreover, exponentiation associates (or brackets) to the right, while the other four main arithmetic operators associate to the left. For example, 2^3^4 means $2^(3^4)$, while $2-3+4$ means $(2-3)+4$. In practice, however, it is often clearer to use explicit parentheses in such expressions, rather than relying on the above rules.

2.3 Standard prelude

Haskell comes with a large number of built-in functions, which are defined in a library file called the *standard prelude*. In addition to familiar numeric functions such as `+` and `*`, the prelude also provides a range of useful functions that operate on lists. In Haskell, the elements of a list are enclosed in square parentheses and are separated by commas, as in `[1,2,3,4,5]`. Some of the most commonly used library functions on lists are illustrated below.

- Select the first element of a non-empty list:

```
> head [1,2,3,4,5]
1
```

- Remove the first element from a non-empty list:

```
> tail [1,2,3,4,5]
[2,3,4,5]
```

- Select the *n*th element of list (counting from zero):